SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (BL2,BL5)

Assessment Tool Weightage: 30%

Duration: 90 Mins **QUESTIONS: 5** **Total Marks:50**

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I MONISH L [1972260035] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student



RUBRICS FOR CALCULATION

Criteria	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Total /10
Weightage	20%	30%	20%	10%	20%	100
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty: *Total Marks Awarded:*

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation environment such as Cisco Packet Tracer, you are required to evaluate the efficiency of data transmission between two nodes. Design a pseudocode algorithm that takes total data received (in bits) and total transmission time (in seconds) as inputs. The algorithm should compute the throughput and display it in bits per second. Include validation to handle cases where time is zero or negative. Incorporate logic to calculate average throughput if multiple readings are given. Finally, suggest how this algorithm can be used to compare different network setups

Answer:

```
BEGIN Throughput_Calculator
  INPUT list of readings, each reading = (data_bits, time_seconds)
  total_throughput = 0
  valid_count = 0
  FOR EACH reading IN readings
    data_bits = reading.data_bits
    time_seconds = reading.time_seconds
    IF time_seconds <= 0 THEN
      DISPLAY "Invalid reading: time must be greater than zero"
      CONTINUE to next reading
    ELSE
      throughput = data_bits / time_seconds   // bits per second
      DISPLAY "Throughput for this reading: " + throughput + " bps"
      total_throughput = total_throughput + throughput
      valid_count = valid_count + 1
    END IF
  END FOR
  IF valid_count > 0 THEN
    average_throughput = total_throughput / valid_count
    DISPLAY "Average throughput: " + average_throughput + " bps"
  ELSE
    DISPLAY "No valid readings to calculate average"
  END IF
END
```

The algorithm processes one or more (data, time) readings. Each reading is validated so that a zero or negative time value is rejected and skipped rather than causing a division-by-zero or a nonsensical negative throughput; only valid readings contribute to the running total. Throughput is computed as data received divided by transmission time, displayed for every valid reading, and an average is produced across all valid readings to smooth out noise from a single measurement. To compare different network setups, the same algorithm can be run once per setup (e.g., once for Setup A's readings and once for Setup B's readings), and the resulting average-throughput values can be placed side by side: the setup with the higher average throughput is transmitting data more efficiently, while consistently low or highly variable per-reading throughput values also flag setups that may need investigation for congestion or link-quality issues.

2. During a simulation experiment, packets are transmitted from a source to a destination, but some may be lost due to congestion or errors. Write a pseudocode algorithm that accepts total packets sent and total packets received. The algorithm should compute the packet loss percentage. Include necessary checks to avoid invalid calculations such as division by zero. Add logic to classify the network condition as Good, Moderate, or Poor. Explain how this algorithm helps in improving network reliability

Answer:

```
BEGIN Packet_Loss_Analyzer
  INPUT total_packets_sent, total_packets_received
  IF total_packets_sent <= 0 THEN
    DISPLAY "Invalid input: total packets sent must be greater than
zero"
    STOP
  END IF
  IF total_packets_received > total_packets_sent THEN
    DISPLAY "Invalid input: received packets cannot exceed sent
packets"
    STOP
  END IF
  packets_lost = total_packets_sent - total_packets_received
  packet_loss_percentage = (packets_lost / total_packets_sent) * 100
  DISPLAY "Packet loss: " + packet_loss_percentage + "%"
  IF packet_loss_percentage < 2 THEN
    condition = "Good"
  ELSE IF packet_loss_percentage <= 5 THEN
    condition = "Moderate"
  ELSE
    condition = "Poor"
  END IF
  DISPLAY "Network condition: " + condition
END
```

The algorithm first guards against invalid inputs: a total of zero (or negative) packets sent would make the percentage calculation undefined (division by zero), and a received count exceeding the sent count signals corrupted input data, so both cases stop execution with a clear error message before any arithmetic is attempted. Packet loss percentage is computed as lost packets divided by sent packets, scaled to a percentage, and the result is classified into Good (under 2% loss), Moderate (2 to 5% loss), or Poor (above 5% loss) bands. This kind of automated classification helps improve network reliability because it turns a raw number into an actionable signal: engineers can set automated alerts on the "Poor" classification to investigate congestion, faulty hardware, or misconfigured routes immediately, rather than manually reviewing loss percentages after problems have already affected users.

3. A network administrator wants to continuously monitor performance metrics using a simulation tool like Cisco Packet Tracer. Write a pseudocode algorithm that periodically collects throughput, delay, and packet loss values. Store these values in a structured format such as arrays or lists. Include a loop to repeat the monitoring process at regular intervals. Add conditions to trigger alerts if any metric exceeds safe limits. Explain how this system can be useful for real-time network management.

Answer:

```
BEGIN Continuous_Network_Monitor
  DECLARE throughput_log = empty array
  DECLARE delay_log = empty array
  DECLARE loss_log = empty array
  SET safe_throughput_min = 1000       // bps
  SET safe_delay_max = 150             // ms
  SET safe_loss_max = 5                // percent
  SET monitoring_interval = 10          // seconds
  WHILE monitoring is active
    throughput = MEASURE_THROUGHPUT()
    delay = MEASURE_DELAY()
    loss = MEASURE_PACKET_LOSS()
    APPEND throughput TO throughput_log
    APPEND delay TO delay_log
    APPEND loss TO loss_log
    IF throughput < safe_throughput_min THEN
      DISPLAY "ALERT: throughput below safe limit"
    END IF
    IF delay > safe_delay_max THEN
      DISPLAY "ALERT: delay above safe limit"
    END IF
    IF loss > safe_loss_max THEN
      DISPLAY "ALERT: packet loss above safe limit"
    END IF
    WAIT for monitoring_interval seconds
  END WHILE
END
```

The algorithm keeps three structured logs (arrays/lists) for throughput, delay, and packet loss so that a full time-series history is preserved for later analysis, not just the most recent reading. A while-loop repeats the measurement-log-check cycle at a fixed interval, and each cycle compares the freshly measured values against predefined safe thresholds, triggering an immediate alert whenever a metric crosses its limit. Because the readings are timestamped implicitly by their position in the log, the stored arrays can later be plotted or averaged to reveal trends (e.g., gradually rising delay) rather than only isolated snapshots. For real-time network management, this design is useful because it lets administrators react to degrading conditions as they happen instead of after a failure is reported by users, and the historical logs support root-cause analysis and capacity-planning decisions after the fact.

4. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

Answer:

```
BEGIN Network_Performance_Summary
  SET total_data_received = 8000        // bits
  SET total_time = 4                    // seconds
  SET total_packets_sent = 200
  SET total_packets_received = 180
  IF total_time <= 0 OR total_packets_sent <= 0 THEN
    DISPLAY "Invalid input: time and total packets sent must be greater
than zero"
    STOP
  END IF
  throughput = total_data_received / total_time         // = 8000 / 4
  packets_lost = total_packets_sent - total_packets_received
  packet_loss_percentage = (packets_lost / total_packets_sent) * 100
  DISPLAY "Throughput: " + throughput + " bps"
  DISPLAY "Packet loss: " + packet_loss_percentage + "%"
  IF throughput > 1500 AND packet_loss_percentage < 10 THEN
    classification = "Efficient"
  ELSE
    classification = "Inefficient"
  END IF
  DISPLAY "Network classification: " + classification
  DISPLAY "Summary -> Throughput: " + throughput + " bps, Packet Loss:
" + packet_loss_percentage + "%, Status: " + classification
END
```

Substituting the given values: throughput = 8000 bits / 4 seconds = 2000 bps, and packets lost = 200 - 180 = 20, giving packet loss = (20 / 200) x 100 = 10%. The validation step confirms neither the time nor the total packets sent is zero before any division is attempted, preventing runtime errors. Applying the classification rule, throughput (2000 bps) does exceed the 1500 bps threshold, but the packet loss (exactly 10%) does not satisfy "less than 10%" -- the condition requires both criteria to hold simultaneously, so the network is classified as Inefficient despite its throughput being acceptable, illustrating why the packet-loss criterion is just as important as throughput when judging overall network performance. The final summary line reports both metrics together with the classification so a reviewer gets the complete picture in one line rather than having to cross-reference two separate outputs.

5. Two different network configurations are tested using a simulation tool, and performance metrics are collected. Write a pseudocode algorithm that takes throughput, delay, and packet loss for both configurations. Compare each metric and assign scores based on performance criteria. Aggregate the scores to determine the better-performing network. Include logic to handle cases where performance is equal. Provide a final output indicating the best configuration with justification.

Answer:

```
BEGIN Compare_Network_Configurations
  INPUT throughput_A, delay_A, loss_A
  INPUT throughput_B, delay_B, loss_B
  score_A = 0
  score_B = 0
  // Higher throughput is better
  IF throughput_A > throughput_B THEN
    score_A = score_A + 1
  ELSE IF throughput_B > throughput_A THEN
    score_B = score_B + 1
  END IF
  // Lower delay is better
  IF delay_A < delay_B THEN
    score_A = score_A + 1
  ELSE IF delay_B < delay_A THEN
    score_B = score_B + 1
  END IF
  // Lower packet loss is better
  IF loss_A < loss_B THEN
    score_A = score_A + 1
  ELSE IF loss_B < loss_A THEN
    score_B = score_B + 1
  END IF
  DISPLAY "Configuration A score: " + score_A
  DISPLAY "Configuration B score: " + score_B
  IF score_A > score_B THEN
    DISPLAY "Configuration A performs better overall"
  ELSE IF score_B > score_A THEN
    DISPLAY "Configuration B performs better overall"
  ELSE
    DISPLAY "Both configurations perform equally overall -- consider
cost or other factors to decide"
  END IF
END
```

Each of the three metrics is compared independently and awards one point to whichever configuration performs better on that metric specifically (higher throughput, lower delay, lower packet loss), so no single metric can dominate the outcome unless it wins outright on all three. The scores are then aggregated and the configuration with more points is declared the better performer; the algorithm explicitly checks for a tie (equal scores) and, rather than arbitrarily picking one side, reports the tie and recommends that a secondary factor such as cost, hardware compatibility, or scalability be used to break it. This scoring approach gives a transparent, metric-by-metric justification for the final

recommendation rather than a single opaque "better" verdict, which makes the result easier to defend when presenting the comparison to stakeholders.